

day17笔记

一、Scheme数据模型

1、Scheme数据模型基础 --- 消息类型

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
from langchain.schema import HumanMessage, AIMessage, SystemMessage
load_dotenv()

llm = ChatTongyi(model='qwen-plus')

# 表示的是用户输入消息
human_msg = HumanMessage(content='你好，请介绍一下你自己。')
print(f'消息类型: {type(human_msg).__name__}')
print(f'消息内容: {type(human_msg.content)}')

# 表示AI给用户的回复
ai_msg = AIMessage(content='你好，我是你的个人智能助理，小美。')
print(ai_msg.content)

# 表示的是需要给AI设定角色和行为规则（系统提示词）
system_msg = SystemMessage(content='你是一个专业的python编程高手，代码编写需要小白都能读懂，代码需要注释')
print(f'系统提示词: {system_msg.content}')
```

2、消息类型使用

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 旧版本导入
# from langchain.schema import HumanMessage, AIMessage, SystemMessage
# 新版本导入（推荐）
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 消息列表
# messages = [
#     SystemMessage(content='你是现代诗人，擅长书写男女之情相关的诗歌'),
#     HumanMessage(content='我叫丁致宇，失恋了，帮我写一手诗歌'),
# ]

messages = [
    SystemMessage(content='你是现代诗人，擅长书写男女之情相关的诗歌'),
    HumanMessage(content='我叫丁致宇，失恋了，给我写一首诗。'),
    AIMessage(content="""
        雨滴在窗上练习倒立行走
        像你名字里未拆封的晴空
    """)
]
```

我数着咖啡渍漫延的疆域
第七圈涟漪吞没了整座城池

衬衫第三颗纽扣突然松动
那是你去年冬天系上的结
如今它滚进沙发缝隙
变成一颗不肯融化的雪

手机屏保还停在樱花季
我们把誓言种成两株并蒂莲
可根系在暗处各自打结
开出背道而驰的春天

现在我要把心跳调成静音模式
任备忘录里的晚安集体叛逃
当所有未发送的早安沉入海底
新酿的月光会浮起另一艘船

"""),

HumanMessage(content='好诗，保持这个风格，再写一首关于丁致宇伤心的诗歌'),

]

输出结果

如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表

result = llm.invoke(input=messages)

print(result.content)

3、消息类型简化

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()
```

llm = ChatTongyi(model='qwen3-max')

消息列表

```
messages = [
    ('system', '你是现代诗人，擅长书写男女之情相关的诗歌'),
    ('human', '我叫丁致宇，失恋了，帮我写一手诗歌'),
    ('ai', ""
     雨滴在窗上练习倒立行走
     像你名字里未拆封的晴空
     我数着咖啡渍漫延的疆域
     第七圈涟漪吞没了整座城池
     ""
    ),
    ('human', '好诗，保持这个风格，再写一首关于丁致宇伤心的诗歌')
]
```

输出结果

如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表

result = llm.invoke(input=messages)

print(result.content)

二、stream流式输出

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 消息列表
messages = [
    ('system', '你是现代诗人，擅长书写男女之情相关的诗歌'),
    ('human', '我叫丁致字，失恋了，帮我写一手诗歌'),
    ('ai', """
        雨滴在窗上练习倒立行走
        像你名字里未拆封的晴空
        我数着咖啡渍漫延的疆域
        第七圈涟漪吞没了整座城池
        """),
    ('human', '好诗，保持这个风格，再写一首关于丁致字伤心的诗歌')
]

# 输出结果
# 如果给模型的是一个消息列表，那么invoke这里传递需要使用关键字参数 input=消息列表
result = llm.stream(input=messages)

for chunk in result:
    print(chunk.content, end='')
```

三、temperature温度

1、temperature可以控制AI输出内容的随机性和创造性，值范围0-2之间

2、特点

- 数值越低，回答越保守、确定，倾向于采用最稳妥、常见的词语回答你
- 数值越高，回答更开放、随机，也愿意使用一些不常用的词语进行组合来回答你

3、示例代码

```
from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max', temperature=0)

result = llm.stream('请用一种非常独特的、非人类的视角，给‘爱情’下一个定义。')

for chunk in result:
    print(chunk.content, end='')

print('\n')
print('-' * 30)
```

```

llm2 = ChatTongyi(model='qwen3-max', temperature=1.2)

result2 = llm2.stream('请用一种非常独特的、非人类的视角，给‘爱情’下一个定义。')

for chunk in result2:
    print(chunk.content, end='')

```

四、给模型添加记忆

1、模型是没有记忆

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

result1 = llm.invoke('我喜欢吃苹果')
print(result1.content)

print('-' * 20)

result2 = llm.invoke('我喜欢吃什么水果? ')
print(result2.content)

```

哇，苹果真是个很棒的选择！🍏 它不仅清脆多汁、酸甜可口，还富含维生素和膳食纤维，对健康特别有益。你最喜欢哪种苹果呢？是红富士的甜美，还是青苹果的爽脆？或者有其他特别钟爱的品种？🍏

我还不知道你喜欢吃什么水果呢！😄

不过，我可以帮你想想——你平时更喜欢甜的、酸的，还是清爽多汁的水果？比如：

- 喜欢甜的：芒果、香蕉、葡萄、荔枝
- 喜欢酸一点的：橙子、柚子、猕猴桃、青苹果
- 喜欢清爽多汁的：西瓜、哈密瓜、梨、草莓

如果你告诉我你的口味偏好，或者最近特别想吃的水果，我可以给你推荐一些哦！你最喜欢哪种水果？

2、添加记忆

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 导入langchain记忆模块
# ConversationChain 和 ConversationBufferMemory 已经被废弃掉了
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# ConversationBufferMemory() 是专门用来给模型添加记忆的（之前需要一个聊天列表需要每次对话把聊天列表传递给模型）
# return_messages=True 作用是为了让咱们查看记忆相关的内容
memory = ConversationBufferMemory(return_messages=True)

# ConversationChain 链，是langchain里面一个重要的模块，把模型和其他模块进行串联
chain = ConversationChain(llm=llm, memory=memory)

```

```

result1 = chain.invoke('我喜欢吃苹果')
print(result1['response'])

print('-' * 20)

result2 = chain.invoke('我喜欢吃什么水果')
print(result2['response'])

print('-' * 20)

# emeory.load_memory_variables({}) 查看记忆相关内容
# history 历史记录
print(emeory.load_memory_variables({}))

```

```
chain = ConversationChain(llm=llm, memory=emeory)
```

哇，真棒！苹果不仅清脆多汁，还富含维生素C和膳食纤维，对身体特别好呢！你喜欢吃哪种苹果？是偏甜的红富士、酸甜适中的嘎啦果，还是特别脆的青苹果？我听说有些人还会把苹果切片蘸花生酱吃，或者做成

你刚刚提到“我喜欢吃苹果”，所以你现在最喜欢吃的水果是苹果呀！🍏

不过水果的世界可丰富啦——除了苹果，你还喜欢别的水果吗？比如香甜的芒果、多汁的西瓜、酸酸甜甜的草莓，还是热带风味的菠萝或百香果？每种水果都有独特的口感和营养，说不定还能发现新的心头好呢！😊

五、Embeddings Models 文本嵌入模型

- 1、作用：可以将字符串作为输入，返回一个浮点型的向量列表
- 2、在NLP中，把数据进行向量化
- 3、示例代码

```

from langchain_community.chat_models.tongyi import ChatTongyi
from dotenv import load_dotenv
# 采用阿里的
from langchain_community.embeddings import DashScopeEmbeddings
load_dotenv()

embed_model = DashScopeEmbeddings(model='text-embedding-v1')

# 把文本向量化 --- 变成浮点数 --- 会计算两个词之间的向量距离（如果语义一样，那么它们的向量距离就比较近）
# 单次转换
# result1 = embed_model.embed_query('女孩脸红了，说："讨厌！"')
# result2 = embed_model.embed_query('女孩愤怒，说："讨厌！"')
#
# print(result1)
# print(result2)

# 批量转换
result3 = embed_model.embed_documents(['人工智能', '机器学习', '西红柿炒鸡蛋', '大模型是如何工作的', 'LLM的原理是什么'])
print(result3)

```